

Unit 3 – Descriptive Statistics

Hands-on Python Programs – Complete Solutions

NAME: MURALI KRISHNAN N

REG NO: 192524090

COURSE CODE: DSA0405

1. Complete Summary of an Employee Dataset

This program loads an employee dataset and reports the number of rows, number of columns, data types, missing values, and a statistical summary.

```
import pandas as pd

# Load the employee dataset (replace with your actual file path)
df = pd.read_csv("employee_data.csv")

print("EMPLOYEE DATASET SUMMARY")
print("=" * 40)

# Number of rows and columns
print("Number of rows   :", df.shape[0])
print("Number of columns:", df.shape[1])

# Data types of each column
print("\nData Types:")
print(df.dtypes)

# Missing values in each column
print("\nMissing Values:")
print(df.isnull().sum())

# Statistical summary (numeric columns)
print("\nStatistical Summary:")
print(df.describe())

# Statistical summary including categorical columns
print("\nFull Summary (all columns):")
print(df.describe(include="all"))
```

Expected Output: Prints row/column counts, a data-type table, a missing-value count per column, and descriptive statistics (mean, std, min, max, quartiles).

2. Frequency Distribution of Students' Grades (Bar Chart)

This program counts how many students received each grade and visualizes the distribution using a bar chart.

```

import pandas as pd
import matplotlib.pyplot as plt

# Sample data: replace with pd.read_csv("students.csv") if you have a file
data = {
    "Student": ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"],
    "Grade":    ["A", "B", "A", "C", "B", "A", "D", "B", "C", "A"]
}
df = pd.DataFrame(data)

# Frequency distribution of grades
freq = df["Grade"].value_counts().sort_index()
print("Frequency Distribution of Grades:")
print(freq)

# Bar chart
plt.figure(figsize=(6, 4))
freq.plot(kind="bar", color="skyblue", edgecolor="black")
plt.title("Frequency Distribution of Students' Grades")
plt.xlabel("Grade")
plt.ylabel("Number of Students")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig("grade_distribution.png")
plt.show()

```

Expected Output: Prints a count of each grade and displays/saves a bar chart showing how many students got each grade.

3. Histogram of House Prices – Normal or Skewed?

This program plots a histogram of house prices and uses the skewness value to decide whether the distribution is approximately normal, right-skewed, or left-skewed.

```

import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import skew

# Sample data: replace with pd.read_csv("house_prices.csv")["Price"]
prices = pd.Series([250000, 300000, 275000, 320000, 500000, 290000,
                    310000, 295000, 305000, 900000, 285000, 260000])

# Histogram
plt.figure(figsize=(6, 4))
plt.hist(prices, bins=8, color="lightgreen", edgecolor="black")
plt.title("Histogram of House Prices")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("house_price_histogram.png")
plt.show()

# Skewness check
sk = skew(prices)
print("Skewness value:", round(sk, 3))

if sk > 0.5:
    print("The data is positively (right) skewed.")

```

```
elif sk < -0.5:
    print("The data is negatively (left) skewed.")
else:
    print("The data is approximately normally distributed.")
```

Expected Output: Displays a histogram of house prices and prints the skewness value with an interpretation (normal, right-skewed, or left-skewed).

4. Box Plot of Salary Data – Detecting Outliers

This program creates a box plot of salary data. Points plotted outside the whiskers represent outliers.

```
import pandas as pd
import matplotlib.pyplot as plt

# Sample data: replace with pd.read_csv("salary.csv")["Salary"]
salary = pd.Series([32000, 35000, 40000, 42000, 38000, 45000,
                    41000, 39000, 150000, 37000, 43000, 5000])

plt.figure(figsize=(5, 5))
plt.boxplot(salary, vert=True, patch_artist=True,
            boxprops=dict(facecolor="lightyellow"))
plt.title("Box Plot of Salary Data")
plt.ylabel("Salary")
plt.tight_layout()
plt.savefig("salary_boxplot.png")
plt.show()

# Identify outliers using IQR
Q1 = salary.quantile(0.25)
Q3 = salary.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = salary[(salary < lower_bound) | (salary > upper_bound)]
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Outliers found:\n", outliers)
```

Expected Output: Shows a box plot of salaries and prints the salary values that lie outside the normal (whisker) range.

5. Detect Outliers in Students' Marks Using IQR Method

```
import pandas as pd

# Sample data: replace with pd.read_csv("marks.csv")["Marks"]
marks = pd.Series([55, 60, 62, 58, 59, 100, 61, 57, 63, 5, 64, 60])

Q1 = marks.quantile(0.25)
Q3 = marks.quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```

print("Q1:", Q1, " Q3:", Q3, " IQR:", IQR)
print("Lower Bound:", lower_bound, " Upper Bound:", upper_bound)

outliers = marks[(marks < lower_bound) | (marks > upper_bound)]
print("\nDetected Outliers:")
print(outliers)

```

Expected Output: Prints Q1, Q3, IQR, the acceptable range, and the list of marks that fall outside that range (the outliers).

6. Remove Outliers from Sales Dataset Using IQR Method

```

import pandas as pd

# Sample data: replace with pd.read_csv("sales.csv")["Sales"]
sales = pd.Series([200, 220, 210, 205, 215, 800, 195, 225, 10, 230, 218])

Q1 = sales.quantile(0.25)
Q3 = sales.quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Original Dataset:")
print(sales.to_list())

# Keep only values within the bounds
cleaned_sales = sales[(sales >= lower_bound) & (sales <= upper_bound)]

print("\nCleaned Dataset (outliers removed):")
print(cleaned_sales.to_list())

print("\nOriginal count:", len(sales))
print("Cleaned count :", len(cleaned_sales))

```

Expected Output: Prints the original sales data, then the cleaned data with outlier values removed, along with the before/after row counts.

7. Histograms and Box Plots Before and After Removing Outliers

```

import pandas as pd
import matplotlib.pyplot as plt

# Sample data
data = pd.Series([200, 220, 210, 205, 215, 800, 195, 225, 10, 230, 218, 208])

Q1 = data.quantile(0.25)
Q3 = data.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

cleaned = data[(data >= lower_bound) & (data <= upper_bound)]

```

```

fig, axes = plt.subplots(2, 2, figsize=(10, 8))

axes[0, 0].hist(data, bins=8, color="salmon", edgecolor="black")
axes[0, 0].set_title("Histogram - Before Removing Outliers")

axes[0, 1].hist(cleaned, bins=8, color="lightgreen", edgecolor="black")
axes[0, 1].set_title("Histogram - After Removing Outliers")

axes[1, 0].boxplot(data, patch_artist=True, boxprops=dict(facecolor="salmon"))
axes[1, 0].set_title("Box Plot - Before Removing Outliers")

axes[1, 1].boxplot(cleaned, patch_artist=True, boxprops=dict(facecolor="lightgreen"))
axes[1, 1].set_title("Box Plot - After Removing Outliers")

plt.tight_layout()
plt.savefig("before_after_outliers.png")
plt.show()

```

Expected Output: Displays a 2x2 grid comparing histograms and box plots of the data before and after outlier removal, showing the distribution becoming tighter and more symmetric.

8. Full Exploratory Data Analysis on Iris / Titanic Dataset

This program performs a complete EDA workflow on the Titanic dataset (seaborn's built-in copy is used here; the Iris dataset works the same way).

```

import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load dataset (Titanic used here; sns.load_dataset("iris") also works)
df = sns.load_dataset("titanic")

# 2. Display dataset information
print("Dataset Info:")
print(df.info())
print("\nShape:", df.shape)
print("\nFirst 5 rows:\n", df.head())

# 3. Find missing values
print("\nMissing Values:")
print(df.isnull().sum())

# 4. Descriptive statistics
print("\nDescriptive Statistics:")
print(df.describe())

# 5. Plot histograms for numeric columns
df.select_dtypes(include="number").hist(figsize=(10, 8), color="steelblue", edgecolor="black")
plt.suptitle("Histograms of Numeric Columns")
plt.tight_layout()
plt.savefig("titanic_histograms.png")
plt.show()

# 6. Plot box plots for numeric columns
numeric_cols = df.select_dtypes(include="number").columns
plt.figure(figsize=(10, 6))
df[numeric_cols].boxplot()

```

```

plt.title("Box Plots of Numeric Columns")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("titanic_boxplots.png")
plt.show()

# 7. Detect outliers using IQR (example: 'fare' column)
Q1 = df["fare"].quantile(0.25)
Q3 = df["fare"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["fare"] < lower_bound) | (df["fare"] > upper_bound)]
print("\nNumber of outliers in 'fare':", len(outliers))

# 8. Remove outliers and save cleaned dataset
df_cleaned = df[(df["fare"] >= lower_bound) & (df["fare"] <= upper_bound)]
df_cleaned.to_csv("titanic_cleaned.csv", index=False)
print("\nCleaned dataset saved as 'titanic_cleaned.csv'")
print("Original rows:", df.shape[0], " Cleaned rows:", df_cleaned.shape[0])

```

Expected Output: Prints dataset info, missing values, and descriptive statistics; displays histograms and box plots for all numeric columns; reports the number of fare outliers detected via IQR; and saves a cleaned CSV file with outliers removed.